Итак, у нас есть датасет: таблица, записи которой описывают взаимодействия между пользователем и товаром, товару соответствует несколько категорий. Установим следующие соглашения:

· t — точка отсчёта (snapshot time), задаётся пайплайном, а не колонкой данных.
· train и test -- 80 % и 20 % записей, качество модели проверяй по holdaut на 20%.
· Таргет(v, i ) = 1, если покупатель v купил товар i в промежутке, и 0 -- иначе.
· horizon — длина окна таргета после t (в этом эксперименте -- 7 дней).
· Окна признаков: n, m, k, q, s, u — гиперпараметры в конфиге.
· Окно фичей: W(t, w) = [t − w, t) (полуоткрытый интервал).
· Все строки считаем транзакциями
· Если знаменатель дроби = 0 → значение признака 0 (зафиксировать это правило).
· Все агрегации — векторно (pandas/polars), без питоновских циклов по строкам.
· Область значений всех параметров -- от 7 до Y, где Y -- все время наблюдения.

Можем выделить несколько групп фич:
Спецификация признаков

Группа 1. Признаки пользователя v

Ключ группировки: visitor. Окно: W(t, n).

user_tx_cnt(v, t, n) — число транзакций пользователя.

user_avg_active_day_gap(v, t, n) — средний интервал в днях между днями активности: множество активных дней A = unique(date(timestamp)), признак = mean(diff(sorted(A))); если |A| < 2 → 0.
user_avg_tx(v, t, n) -- среднее число транзакций пользователя в неделю, за последние n дней.

Группа 2. Признаки товара i

Ключ группировки: itemid. Окно: W(t, m).

item_tx_cnt(i, t, m) — число транзакций товара.
item_avg_active_day_gap(i, t, m) — по аналогии с 1.4.
item_repeat_purchase_rate(i, t, m) — число пользователей, купивших товар i хотя бы дважды в разные дни внутри окна.
item_avg_tx(i, t, m) -- среднее число транзакций с товаром в неделю, за последние m дней.

Группа 3.1 Признаки категории товара c
Ключ группировки: item_cat. Окно: W(t, k).
cat_tx_cnt(c, t, k) число транзакций с категорией за последние k дней

item_avg_tx(c, t, k) -- среднее число транзакций с категорией в неделю, за последние m дней.

Группа 3.2 Признаки категории покупателя c
По аналогии с признаками товара. Параметр - x. Ключ группировки: item_cat. Окно: W(t, x).

Группа 4. Признаки пары (пользователь v, товар i)
Ключ группировки: (visitor, itemid). Окно: W(t, q).
vi_tx_cnt(v, i, t, q) — число транзакций с данной парой.

Группа 5.1 Признаки пары (пользователь v, категория товара c)
Ключ группировки: (visitor, category). Окно: W(t, s).
vc_tx_cnt(v, c, t, s) --- число транзакций с данной парой.

Группа 5.2 Признаки пары ( товар i, категория покупателя c)
Ключ группировки: (itemid, visitor_cat). Окно: W(t, z).
ic_tx_cnt(v, c, t, z) --- число транзакций с данной парой.

Группа 6. Признаки пары (товар i, товар j)
Ключ группировки: (itemid_i, itemid_j). Окно: W(t, u).
ii_co_purchase(i, j, t, u) — бинарный: 1, если ≥ 2 различных пользователя купили оба товара внутри окна, иначе 0.
Algoritm short_grid_search
Эксперимент 1.
Для каждой группы фич будет проведен эксперимент. необходимо построить группу моделей. Модели группы будут обучены по всем вышеуказанным признакам, но с разными параметрами. Параметры всех групп, кроме выделенной исследуемой, будут выбраны как среднее из области определения параметра. Для параметра исследуемой группы будет рассмотрено 5 значений, для каждого из них обучаем отдельную модель. Определяем для каждого из параметров оптимальное значение, далее создаем таблицу exp_1.csv и вносим в нее полные названия признаков и оптимальные значения параметров.
Эксперимент 2.
Сначала выбираем оптимальные (выбранные экспериментально на предыдущем шаге) значения всех параметров. Для этих параметров обучаем модель на всех признаках . Далее для каждой группы фич строим одну отдельную модель: эта модель обучается на всех фичах, кроме фич указанной группы. Параметры все еще берем оптимальные. Создаем таблицу exp_2.csv, в ней также будут все признаки, пометь 0 те, удаление которых не делает модель хуже, и 1 -- остальные. (Считаем, что модель стала хуже, если ROC уменьшилось хотя бы на 0.01, или если ROC изменилась менее чем на 0.01, а PR уменьшилось хотя бы на 0.005)
Эксперимент 3.
Далее рассматриваются только значимые признаки -- те, после удаления которых модель стала хуже. Каждому из оставшихся признаков присвоим новый, каждому отдельный, параметр, и для каждого признака построим отдельную модель. Для параметра исследуемого признака рассмотрим 5 значений, выбранных через равные

промежутки из области значений. Остальные параметры будут установлены как среднее из области значений. Таким образом, для каждого параметра будет выбрано оптимальное значение. Создаем таблицу exp_3.csv, в ней будут все значимые признаки и оптимальные значения их параметров.

Финал: устанавливаем каждому из параметров оптимальное значение, обучаем модель. подробно опиши, какие признаки с какими параметрами используешь для каждой модели. все обученные модели внеси в таблицу models.csv и сравни по метрикам. Конец алгоритма

Создай таблицу final_models_diff_versions.csv и запиши в нее данные об этой финальной модели: признаки, параметры, ROC и PR. Выгружать пока не надо.

Посчитанные признаки используй в дальнейшей работе.

Увеличь horizon вдвое, проведи эксперименты 1-3 и финал, занеси данные о финальной модели в таблицу final_models_diff_versions.csv.

Измени правила сеточного поиска: теперь будет по 10 значений через равные промежутки, horizon по прежнему 14, все остальной как раньше, проведи эксперименты 1-3 и финал, занеси данные о финальной модели в таблицу final_models_diff_versions.csv.

Добавь еще 4 эксперимент: сеточный поиск с шагом 10 в небольшой (+-20) окрестности оптимальных значений параметра. То есть, если оптимальное значение параметра -- a, ты рассмотришь значения a - 20, a - 10, a, a + 10, a + 20. Все остальное как раньше (в сеточном поиске по 10 значений, horizon по прежнему 14). Проведи эксперименты 1-3 и финал, занеси данные о финальной модели в таблицу final_models_diff_versions.csv.

Замени сеточный поиск на стохастический с фиксированным шагом 10 и терпением 10. Все остальное как раньше, но без эксперимента 4. Проведи эксперименты 1-3 и финал, занеси данные о финальной модели в таблицу final_models_diff_versions.csv.

Замени стохастик с фиксированным шагом 10 и терпением 10 на стохастик с адаптивным шагом, то есть стохастический поиск будет выполняться с адаптивным шагом: от 80 до 5, старт -- +- 80, после исчерпания обоих направлений делим шаг пополам (если он был >5). Все остальное как раньше, но без эксперимента 4. Проведи эксперименты 1-3 и финал, занеси данные о финальной модели в таблицу final_models_diff_versions.csv.

Теперь изменим структуру алгоритма. Попробуй выставить параметры на среднее значение в промежутке значений, и сразу провести эксперименты 2 и 3 и финал. Занеси данные о финальной модели в таблицу final_models_diff_versions.csv.

Теперь попробуй сразу присвоить всем имеющимся изначально признакам различные параметры (каждому признаку отдельный параметр). Проведи стохастический поиск с

адаптивным шагом и терпением 20. Занеси данные о финальной модели в таблицу final_models_diff_versions.csv.